Software Requirements Specification

for

NoMem

Version 1.3

Prepared by

Team Number: 5

Kaustubh Bahl (Lead)	**B200833CS**
Sanil Mishra	**B200057CS**
Daruri Kaustubh	**B200825CS**
Gigil James	**B200771CS**
Bharath Sajan	**B200016CS**
Ananya Saha	**B200789CS**

Project Manager: ***Ms Renya Nath N***

Project Client: ***Dr. P N Bala Subramaniam***

Course: **CS4096D Software Engineering Laboratory**

Date: **23/03/2023**

This template is based on the one available from GMU site by Dr. Rob Pettit. Modifications specific to NITC are made and will be used for academic purpose only.

1 Introduction 4
1.1 Document Purpose 4
1.2 Product Scope 4
1.3 Intended Audience and Document Overview 4
1.4 Definitions, Acronyms, and Abbreviations 5
1.5 Document Conventions 5
1.6 References and Acknowledgments 5
2 Overall Description 5
2.1 Product Overview 5
2.2 Product Functionality 6
2.3 Design and Implementation Constraints 7
2.4 Assumptions and Dependencies 7
3 Specific Requirements 7
3.1 External Interface Requirements 7
3.1.1 User Interfaces 7
3.1.2 Hardware Interfaces 9
3.1.3 Software Interfaces 9
3.2 Functional Requirements 9
3.3 Use Case Model 10
3.3.1 Generate password for a new account – U1 10
3.3.2 Select an existing account – U2 11
3.3.3 View existing accounts' list – U3 12
3.3.4 Search for an account – U4 12
3.3.5 Sort accounts list alphabetically – U5 13
3.3.6 View password – U6 14
3.3.7 Update password – U7 15
3.3.8 Delete account – U8 16
3.3.9 Copy password to clipboard – U9 16
3.3.10 Export account details – U10 17
3.3.11 Import account details – U11 18
3.3.12 Unlock the app - U12 19
3.3.13 View account's details - U13 19
4 Other Non-functional Requirements 20
4.1 Performance Requirements 20
4.2 Safety and Security Requirements 20
4.3 Software Quality Attributes 21

4.3.1 Reliability 21
4.3.2 Maintainability 21
4.3.3 Testability 21
4.3.4 Portability 21
4.3.5 Cost-effectiveness 21
4.3.6 Correctness 21
Appendix A - Activity Log 22

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Kaustubh Bahl Sanil Mishra Daruri Kaustubh Gigil James Bharath Sajan Ananya Saha	Initial version	15/03/2023
1.1	Kaustubh Bahl Sanil Mishra Daruri Kaustubh Gigil James Bharath Sajan Ananya Saha	grammatical corrections, fixed table of contents, defined ‘strong passwords’ in 2.2, minor changes to 3.3.2, 3.3.4, 3.3.8	16/03/2023
1.2	Kaustubh Bahl Sanil Mishra Daruri Kaustubh Gigil James Bharath Sajan Ananya Saha	Added definitions to section 1.4 for clarity; Added import & export features – changes in sections 2.1 (flowchart), 3.2, 3.3; Elaborated GUI in section 3.1; Changes to sections 4.1, 4.2, and 4.3 for more clarity.	16/03/2023
1.3	Kaustubh Bahl Sanil Mishra Daruri Kaustubh Gigil James Bharath Sajan Ananya Saha	Changes based on SRS Review; added app lock functionality; added backup reminder functionality; added UI screens;	23/03/2023

1 Introduction

This project is NoMem, a password manager application intending to manage and organise passwords securely. This section provides a basic overview of the project's aim, significance, and utility of the software product to be formed, along with focusing on the targeted set of users.

1.1 Document Purpose

The primary purpose of this document is to provide a detailed analysis of the Password Manager project. This document reflects on both the functional and non-functional requirements needed for achieving the objectives of the product. The password manager offers users a simple yet effective solution to keep their passwords safe and easily accessible. The document also focuses on the external interface requirements involving both hardware and software interfaces. It also emphasises the system's functionalities and the interaction between the system and the users with the help of use case diagrams.

1.2 Product Scope

NoMem provides a more productive environment for the users, reducing the need to remember multiple complex passwords. This includes generating strong passwords using industry-standard hashing algorithms. The password manager would also provide a user-friendly interface for quickly adding, modifying, and deleting passwords. NoMem thus delivers a comprehensive solution for securely managing passwords for personal and professional use.

1.3 Intended Audience and Document Overview

The document is intended to serve several groups of audience members.

- Firstly, the SRS will be referenced by the system designers to create the design of the application.
- Secondly, the developers and testers shall view to ensure the requirements have been met.
- Thirdly, the project's client shall review the SRS to understand the primary product functionalities.
- The application maintenance staff can also review the document for future modifications to get clarity on the present functionality.

1.4 Definitions, Acronyms, and Abbreviations

S. No	Abbreviation/Term	Definition(s)
1	Account	An entity in the system corresponding to the account for which a password needs to be generated.
2	Account's details	Attributes associated with the account entity, viz. domain, username, version number, and length.
3	NoMem	Name of the app
4	SRS	Software Requirements Specifications
5	User	Any person using the app

1.5 Document Conventions

This document follows the IEEE formatting requirements.

1.6 References and Acknowledgments

- IEEE. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998

2 Overall Description

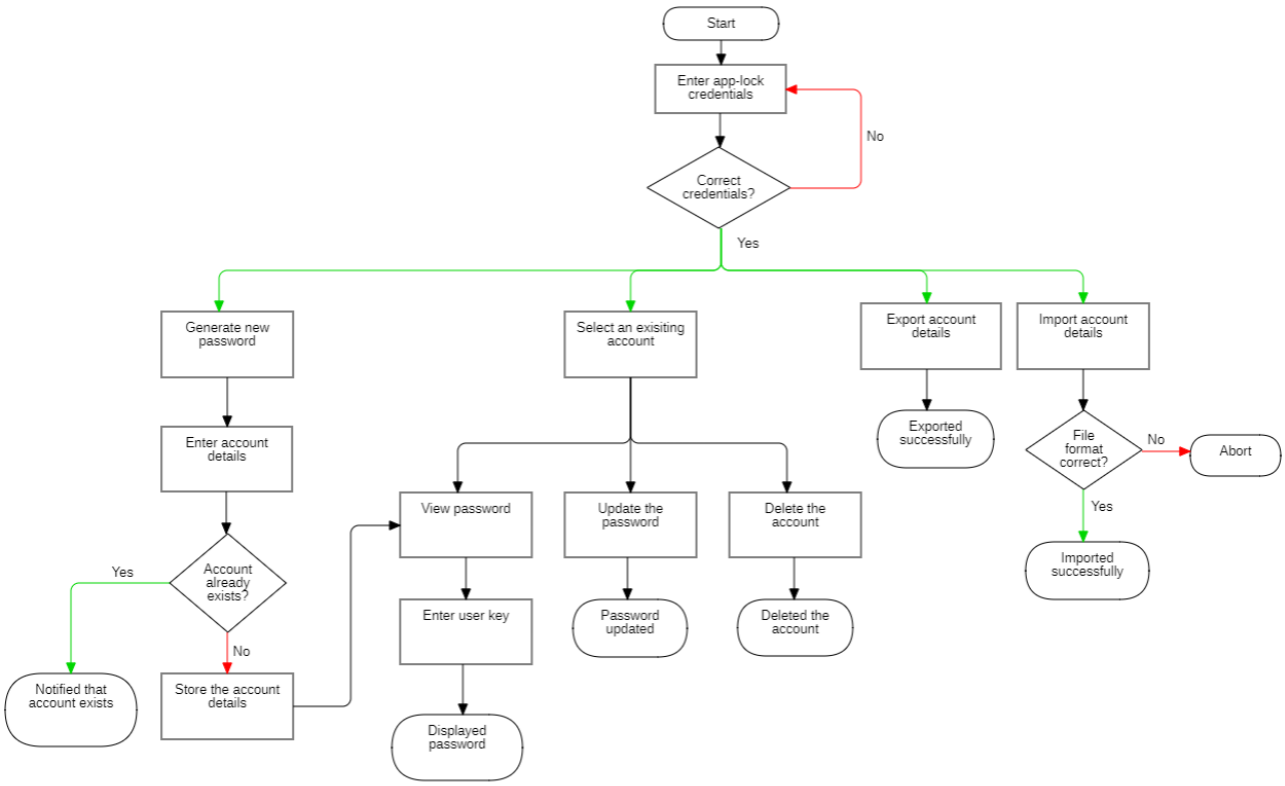
2.1 Product Overview

The Password Manager is a secure password management application that generates secure passwords from a single user-inputted key which is hashed with other additional details, such as website/app domain name, username, and version number, etc., to ensure that each password is strong. The ultimate feature of the application is that it only requires the user to remember their user key for every password generation and doesn't store any sensitive information.

The user can generate a password from existing accounts or create a new account. To create a new account, the user enters account details related to the web/app domain and customises the password generation settings if required.

If the account exists, the user selects it and enters the user key to generate the corresponding account's secure password. When updating the password for an account, the user only needs to update/ increment the version number of the associated account, which will generate a new secure password.

The application is not connected to the internet to prevent potential security risks. Additionally, the application provides local backup and recovery features. Also, mandatory user authentication to access the application is required, providing an extra layer of security to protect it from unauthorised access. For this, the in-built app lock of the OS is used.



2.2 Product Functionality

The Password Manager will:

a) Generate strong and non-guessable passwords for the user. Strong passwords have at least one character belonging to all character classes – lowercase letters, uppercase letters, digits, and special characters.
b) Permit local data backup of account details, including giving a reminder to do so on modification of any account's details.
c) Store generated accounts' details (associated with password) securely in a local database.

The Password Manager will allow the users to:

d) Select their user key to be inputted for password generation.
e) Specify the password attributes, such as password length, if they wish to.
f) View accounts' details and passwords.
g) Browse all the accounts and search for a particular one.
h) Update the password for any account.

2.3 Design and Implementation Constraints

- The application must ensure the highest level of security for the password generation algorithm.
- The hashing algorithm designed/chosen should be collision resistant to reduce the probability of passwords being repeated.
- The hashing algorithm must be deterministic, i.e., given the same inputs (account details and user key) same output (password) must be generated every time it runs.
- Users must enter their correct user key to generate the correct password.
- Any issuing/ updation/ deletion of the account's details must be updated in the database.
- The application should be compatible with Android 10 and above operating system versions.

2.4 Assumptions and Dependencies

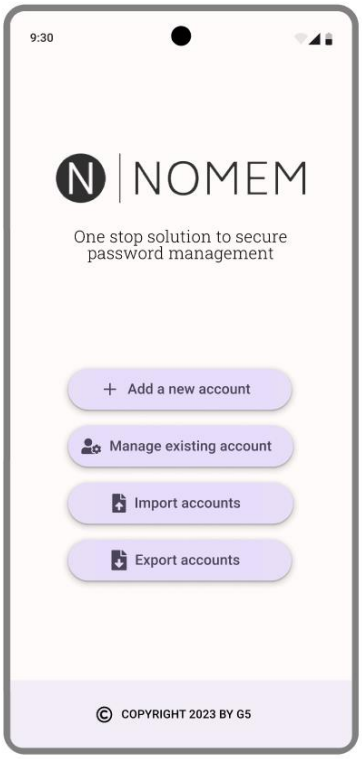
- The application assumes that the user will input accurate and valid information, such as their user key and additional details, such as domain names or usernames.
- The user does not share his user key with anyone.
- Apart from the core framework, some other dependencies of Flutter might be used.
- The app will utilise the OS-provided app lock to authenticate the user.

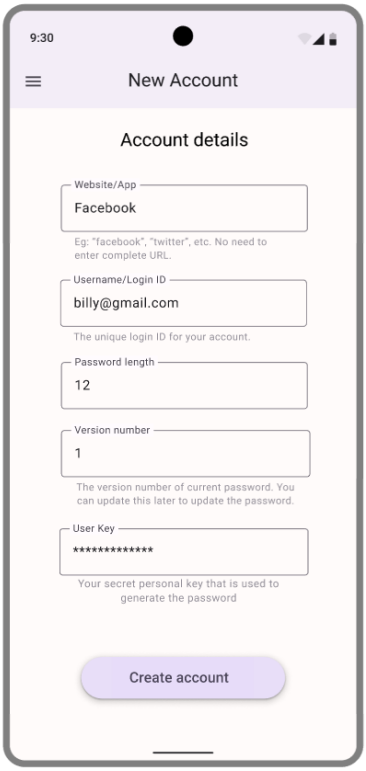
3 Specific Requirements

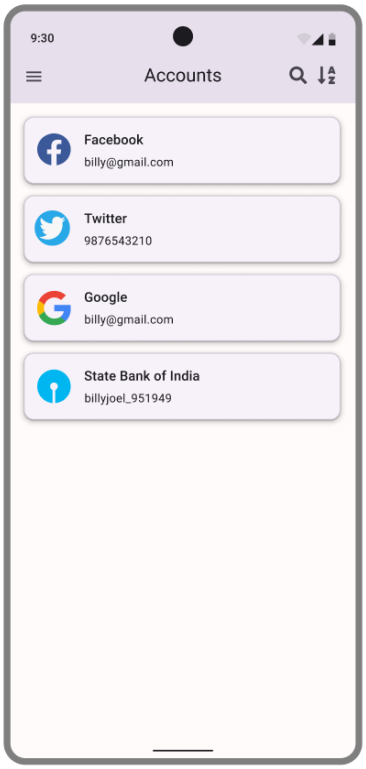
3.1 External Interface Requirements

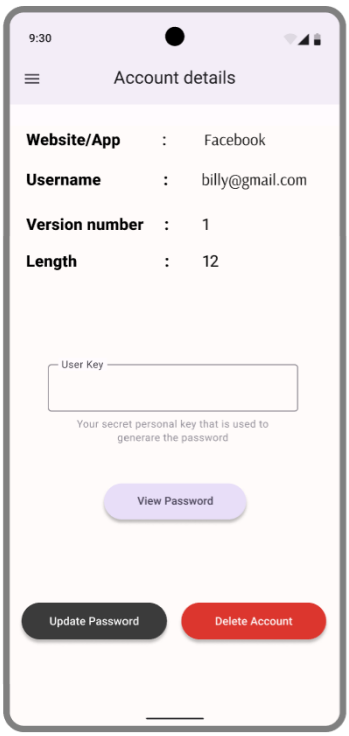
3.1.1 User Interfaces

- The design or layout of every interface will be intuitive, easily understandable, and interactive to the user.
- Upon opening the application, the user will be on the home screen.
- From the home screen, the user may opt to go to the password generation screen, view the accounts list screen, or import or export account details.
- On the password generation/new account screen, the user can create a new account and view the password after filling in the required input in the form.
- On the accounts list screen, all the accounts are listed, and the user can search and sort.
- On an account screen, the user can view the account details and further view the password, update the password, or delete the account.
- The background, graphics, and font in the screen layout will be adequate and well-organized, making the content readable.
- The user can perform all the required tasks following the proper guidelines.


9:30
NOMEM
One stop solution to secure password management
Add a new account
Manage existing account
Import accounts
Export accounts
© COPYRIGHT 2023 BY G5


9:30
New Account
Account details
Website/App
Facebook
Eg: "facebook", "twitter", etc. No need to enter complete URL.
Username/Login ID
billy@gmail.com
The unique login ID for your account.
Password length
12
Version number
1
The version number of current password. You can update this later to update the password.
User Key
****
Your secret personal key that is used to generate the password
Create account


9:30
Accounts
Facebook
billy@gmail.com
Twitter
9876543210
Google
billy@gmail.com
State Bank of India
billyjoel_951949


9:30
Account details
Website/App : Facebook
Username : billy@gmail.com
Version number : 1
Length : 12
User Key
Your secret personal key that is used to generare the password
View Password
Update Password
Delete Account

9:30
Account details
Website/App : Facebook
Username : billy@gmail.com
Version number : 1
Length : 12
Tap the password to copy
D%x7fGio5V#t
Close
View Password
Update Password
Delete Account

3.1.2 Hardware Interfaces

The minimum hardware requirements that the mobile must satisfy:

- Processor: Single-core 1GHz processor.
- RAM: 4 GB
- Display: 720 x 1280 pixels

3.1.3 Software Interfaces

Software	Description
Operating System	Android 10 and above Android platforms will be supported.
Frontend and Backend	Flutter is an easy-to-learn and reliable framework that can be used to develop high-quality mobile apps easily and quickly.

3.2 Functional Requirements

F1: The system shall allow the user to authenticate via OS-provided App Lock.

F2: The system shall allow the user to generate a password for a new account and store the new account's details in the local database, which can be used to regenerate the same password again, given the same user key.

F3: The system shall allow the user to select an existing account.

F4: The system shall allow the user to view the list of existing accounts.

F5: The system shall allow the user to search for an account from the accounts list.

F6: The system shall allow the user to sort the accounts list alphabetically.

F7: The system shall allow the user to view the password for the selected account.

F8: The system shall allow the user to copy the displayed password to the clipboard.

F9: The system shall allow the user to update the password for the selected account.

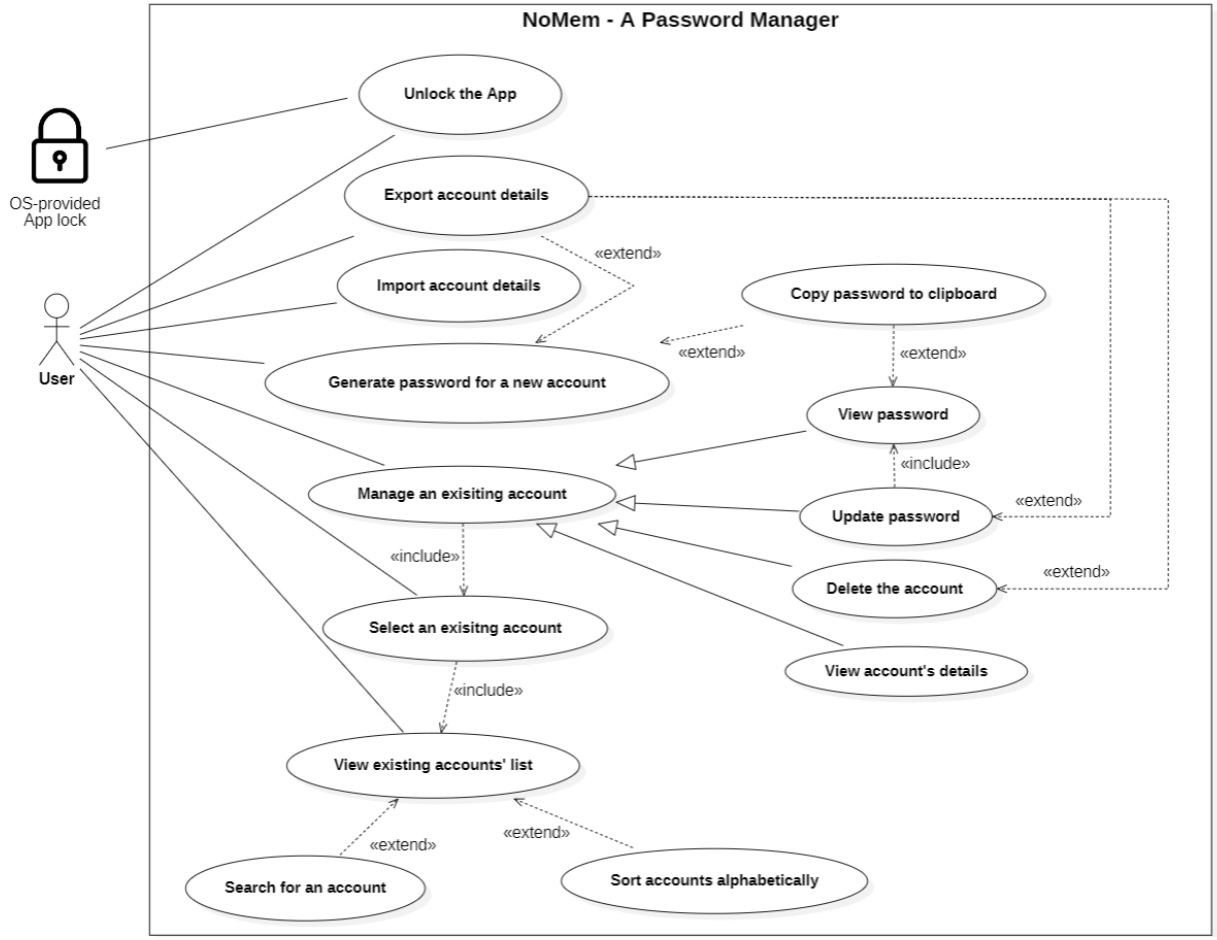
F10: The system shall allow the user to delete the selected account.

F11: The system shall allow the user to view the details of the selected account.

F12: The system shall allow the user to import details of accounts from a file on the device.

F13: The system shall allow the user to export details of accounts to a file on the device.

3.3 Use Case Model



3.3.1 Generate password for a new account – U1

Author – Daruri Kaustubh

Purpose - Generate a password for a new account and add details of the new account to the system.

Requirements Traceability – F2

Priority - High

Preconditions –

1. The app is unlocked.

Postconditions –

1. A password for the account is generated and shown to the user.
2. The account details are stored in the system.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –

    1. The user enters the user key, domain, and username in the form displayed on the screen. The version number and the password lengths have default values, which can be changed.
    2. The user clicks on submit button.
    3. The account's details are stored in the local database.
    4. System gives the user choice of exporting, and appropriate actions are taken (U10).
    5. The generated password is displayed as a pop-up message.
    6. The user dismisses the message.
    7. System clears the form.

2. Alternative Flow –

    1. The user submits a form with a domain and a username that matches an existing account's domain and username.
    2. System shows a message informing that an account already exists.
    3. The system clears the form.

Includes – None

3.3.2 Select an existing account – U2

Author – Sanil Mishra

Purpose - Select an account from a list of existing accounts.

Requirements Traceability – F3

Priority - High

Preconditions –

1. The app is unlocked.
2. A list of existing accounts is displayed on the screen.

Postconditions –

1. The desired account is successfully selected.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –

    1. User selects an account from the accounts list.
    2. System selects the account.

Includes – U3

3.3.3 View existing accounts' list – U3

Author – Ananya Saha

Purpose - View a list of the accounts.

Requirements Traceability – F4

Priority - High

Preconditions –

1. The app is unlocked.

Postconditions –

1. The system displays a list of existing accounts.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. System displays the list of all the accounts' domains and usernames present in the database.
2. Alternative Flow –
    1. System displays the list of the accounts' domains and usernames of only the accounts satisfying the search text.

Includes – None

Notes/Issues – The accounts list might be empty without accounts.

3.3.4 Search for an account – U4

Author – Gigil James

Purpose - Search for an account based on username or domain.

Requirements Traceability – F5

Priority - Medium

Preconditions –

1. The app is unlocked.
2. The system is displaying a list of existing accounts.

Postconditions –

1. The account list contains only the accounts that filter through the search query.

Actors – User

Extends – U3

Flow of Events -

1. Basic Flow –
    1. User enters text in the search field.
    2. The system returns the list of all the accounts with the search text as a substring in the domain or the username.
    3. The system updates the accounts list with the list mentioned above.

Includes – None

Notes/Issues –

1. The account list may be empty (both before and after searching).
2. The account list is updated with each character entered (doesn't wait for a submit button).

3.3.5 Sort accounts list alphabetically – U5

Author – Kaustubh Bahl

Purpose - Sorts the account list alphabetically.

Requirements Traceability – F6

Priority - Low

Preconditions –

1. The app is unlocked.
2. The system is displaying a list of all the accounts.

Postconditions –

1. The accounts list gets sorted alphabetically based on the domain name. The accounts with the same name get sorted alphabetically based on the username.

Actors – User

Extends – U3

Flow of Events -

1. Basic Flow –
    1. User selects the sort alphabetically option.
    2. System sorts the accounts list alphabetically based on the domain names. The accounts with the same domain name get sorted based on the username.
    3. The system updates the accounts list with the list mentioned above.

Includes – None

Notes/Issues –

1. The account list may be empty (both before and after sorting).
2. The change in the order of accounts is not reflected in the local database. Returning to the same screen later won't display the accounts in alphabetical order necessarily.

3.3.6 View password – U6

Author – Bharath Sajan

Purpose - To view the password for an account.

Requirements Traceability – F7

Priority - High

Preconditions –

1. The app is unlocked.
2. An account has been selected.

Postconditions –

1. The password for the account is displayed on the screen.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. User enters the user key.
    2. User selects the view password option.
    3. System displays the password for the account in a pop-up message.
    4. The user dismisses the message.
    5. The system clears the user-key field.

Includes – None

Notes/Issues - None

3.3.7 Update password – U7

Author – Ananya Saha

Purpose - To update the password by incrementing the version number.

Requirements Traceability – F9

Priority - High

Preconditions –

1. The app is unlocked.
2. An account is selected.

Postconditions –

1. The version number for the account gets incremented in the local database.
2. The old password on the screen is replaced by the new one if present.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. System displays the account details (domain, username, version number, length, etc.).
    2. User selects the update password option.
    3. The system increments the account's version number and updates the locally stored details.
    4. If the old password is on the screen, the system replaces it with the new one.
    5. User is notified that the password has been updated.
    6. System gives the user choice of exporting, and appropriate actions are taken (U10).

Includes – U6

Notes/Issues – None

3.3.8 Delete account – U8

Author – Gigil James

Purpose - To delete an account's details.

Requirements Traceability – F10

Priority - Low

Preconditions –

1. The app is unlocked.
2. An account is selected.

Postconditions –

1. The account details get deleted.
2. User is redirected to the accounts list screen.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. The user selects the delete account option.
    2. System deletes the account details from the local database.
    3. The system informs the user that the account has been deleted.
    4. System gives the user choice of exporting, and appropriate actions are taken (U10).
    5. User is directed to the accounts list screen.

Includes – None

Notes/Issues - None

3.3.9 Copy password to clipboard – U9

Author – Sanil Mishra

Purpose - The password gets copied to the clipboard.

Requirements Traceability – F8

Priority - Low

Preconditions –

1. The app is unlocked.
2. The password for the account is displayed on the screen

Postconditions –

1. The password of the account gets copied to the clipboard.

Actors – User

Extends – U1, U6

Flow of Events -

1. Basic Flow –
    1. The user taps the password.
    2. The password gets copied to the clipboard.
    3. The system notifies the user that the password has been copied.

Includes – None

Notes/Issues – None

3.3.10 Export account details – U10

Author – Kaustubh Bahl

Purpose - To export the details of all the accounts into a file.

Requirements Traceability – F13

Priority - Medium

Preconditions –

1. The app is unlocked.

Postconditions –

1. An encrypted file containing the details of all the accounts will be downloaded to the local device.

Actors – User

Extends – U1, U7, U8

Flow of Events -

1. Basic Flow –
    1. User clicks on the export option.
    2. An encrypted file with account details is downloaded.
    3. System informs the user of successful export.
2. Alternative Flow –
    1. On modifying the accounts' details, the system gives a prompt, giving a choice of exporting or dismissing the message.
    2. User selects the export option.
    3. An encrypted file with account details is downloaded.
    4. System notifies the user of successful export.

Includes – None

Notes/Issues - The downloaded file might be empty if there are no accounts in the local database

3.3.11 Import account details – U11

Author – Daruri Kaustubh

Purpose - To import the account details from a file.

Requirements Traceability – F12

Priority -Medium

Preconditions –

1. The app is unlocked.

Postconditions –

1. The account details from the file are added to the application's local database.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. User clicks on the import option.
    2. User selects a file with the correct file format (must match the file format used by the app) from the device.
    3. The account details in the file are added to the application's local database.
    4. The system notifies the user of successful import.
2. Exception flow -
    1. User clicks on the import option on the home screen.
    2. User selects a file of a different format than specified above.
    3. The User is notified that the file format is incorrect.

Includes – None

Notes/Issues – No accounts will be added if the file is empty.

3.3.12 Unlock the app - U12

Author – Bharath Sajan

Purpose - To authenticate the user using the OS-provided app lock.

Requirements Traceability – F1

Priority - Medium

Preconditions – None

Postconditions –

1. The app is unlocked.

Actors – User, OS-provided App Lock

Extends – None

Flow of Events -

1. Basic Flow -
    1. User opens the app.
    2. System displays a prompt asking the user to authenticate using the OS-provided app lock.
    3. User enters the correct credentials.
    4. The app gets unlocked.
2. Exception Flow –
    1. User opens the app.
    2. System displays a prompt asking the user to authenticate using the OS-provided app lock.
    3. User enters the incorrect credentials.
    4. The OS-provided app lock informs the user about the wrong credentials and asks the user to try again.

Includes – None

Notes/Issues - None

3.3.13 View account's details - U13

Author – Kaustubh Bahl

Purpose - To view the details of the selected account.

Requirements Traceability – F11

Priority - Medium

Preconditions –

1. The app is unlocked.
2. An account is selected.

Postconditions –

1. The details of the selected account are displayed on the screen.

Actors – User

Extends – None

Flow of Events -

1. Basic Flow –
    1. System displays the details of the selected account on the screen.

Includes – None

Notes/Issues - None

4 Other Non-functional Requirements

4.1 Performance Requirements

The following performance requirements are essential for a positive user experience:

- Time: The application should be able to perform all the specified operations fast. The response time should not exceed 3 seconds.
- Storage space: The app uses a local database. The total storage space usage should not exceed 150 MB under the assumption of the user using under 60 accounts.

4.2 Safety and Security Requirements

- The system generates passwords using irreversible hashing algorithms, and since it doesn't store the user key or the passwords anywhere, the passwords are safe from being leaked.
- A collision-resistant algorithm will be chosen to reduce the probability of repeated passwords.
- The account details will be stored locally on the device in an encrypted form to keep the account details safe from leakages.
- The user has the option to export the account details and thus have a local backup. The user may store this wherever they feel safe. In the case of device damage, loss, or the app getting uninstalled, the user can import the account details from the file.
- The app won't use the internet for any of its services.

- To prevent passwords from being leaked from the user's end, users are advised not to write down the generated passwords or the user keys anywhere.

4.3 Software Quality Attributes

4.3.1 Reliability

The system will be reliable. It won't hang or crash. Users will be able to perform the required operations smoothly.

4.3.2 Maintainability

Readable and simple structured code and a well-designed system will ensure that maintainers can easily understand the code. Flutter follows the 'single responsibility principle,' making understanding and debugging code easier. The 'Hot Reload' feature of Flutter allows making changes quickly and seeing the result without rebuilding the app every time. This makes adding features, fixing bugs, etc., easy.

4.3.3 Testability

Flutter makes testing easy by providing various dependencies for unit, widget, and integration testing.
Google Play Store allows developers to create releases on three testing phases – internal, closed, and open testing.

4.3.4 Portability

The system will be initially deployed only on devices running Android 10 and above Android versions. In the future, the app can be extended to web, iOS, Linux, Windows, and Google Fuchsia platforms easily as Flutter is a cross-platform app development framework and thus uses the same codebase.

4.3.5 Cost-effectiveness

Flutter is open-source and free to use. The only cost is the deployment cost. To deploy any number of apps on the Google Play Store, only a one-time fee needs to be paid.
This app will be available for everyone for free.

4.3.6 Correctness

The application will be correct in terms of adhering to the functional requirements and will exhibit all the necessary functionalities.

Appendix A - Activity Log

Contributions:

- Kaustubh Bahl – Section 2.1 (flowchart), 3.1.1 (text content), 3.1 (home screen), 3.1.2, 3.1.3, 3.3, 3.3.5, 3.3.10, 3.3.13
- Sanil Mishra – Section 1, 1.1, 1.2, 1.3, 3.1.1 (accounts list screen), 3.3.2, 3.3.9
- Daruri Kaustubh – Section 3.1.1 (account screen), 3.2, 3.3.1, 3.3.11
- Gigil James – Section 3.1.1 (account screen with password), 3.3.4, 3.3.8, 4
- Bharath Sajan – Section 2.1 (text content), 2.2, 2.3, 2.4, 3.1.1 (new account screen), 3.3.6, 3.3.12
- Ananya Saha – Section 1.4, 1.5, 1.6, 3.3.3, 3.3.7

Team Meets:

- 13.03.2023, 11 AM – 12 Noon: Members discuss Sections 2 and 3.
- 13.03.2023, 10 PM – 11 PM: Discussion between the team and the client about the project requirements and functionalities.
- 14.03.2023, 9 PM – 12 Midnight: Team meet to prepare the SRS 1.0.
- 15.03.2023, 11 AM – 12 Noon: Finishing the SRS 1.0.
- 16.03.2023, 9 AM – 9:30 AM: Minor updates to SRS 1.0 to prepare SRS 1.1
- 16.03.2023, 9 PM – 11 PM: Making changes, preparing SRS 1.2
- 22.03.2023, 10:30 AM – 11 AM: Meet between the client, the manager, and the team to discuss changes mentioned in the SRS review.
- 22.03.2023, 8 PM – 10 PM: Team meet to discuss changes based on the feedback received in SRS Review and client meet.
- 23.03.2023, 9 AM – 12 Noon: Team meet to make the required changes to make SRS 1.3 and a final review.